



SYN Flooding using SCAPY and Prevention USING IPTABLES

DOS (Denial of Service) attacks against Web services make them unavailable for legitimate users, affecting the website owner's potential business. These involve intentional consumption of network, CPU and memory resources. In this article, I will demonstrate how to do a SYN flood using the SCAPY framework, along with other preventive measures.

Over time, DOS attacks have become more complicated, concealing malicious client requests as legitimate ones. Also, a distributed approach, the DDOS (Distributed Denial of Service) is now being adopted, which involves generating multiple requests to create a flood scenario. One type of DDOS flood attack is the TCP SYN queue flood.

A SYN queue flood attack takes advantage of the TCP protocol's 'three-way handshake'. A client sends a TCP SYN (S flag) packet to begin a connection to the server. The target server replies with a TCP SYN-ACK (SA flag) packet, but the client does not respond to the SYN-ACK, leaving the TCP connection 'half-open'. In normal operation, the client should send an ACK (a flag) packet followed by the data to be transferred, or an RST reply to reset the connection. On the target server, the connection is kept open, in a 'SYN_RECV' state, as the ACK packet may have been lost due to network problems. In a DDOS, multiple attackers make many such half-connections to the target server, in a storm of requests. When the server's SYN buffer is full with half-open TCP connections, it stops accepting SYN connections, thus resulting in denial of service to legitimate clients.

Such DDOS attacks are generally carried out using 'bot-nets' of other compromised systems across the Internet, which through back-doors and Trojans are directed to send artificial SYN flood traffic to targeted servers. To defend against such attacks, a strong monitoring system is required, as there is a very fine line between legitimate and fake clients. SYN queue flood attacks can be mitigated by tuning the kernel's TCP/IP parameters.

In this article, to simulate a DDOS, I will generate SYN flood packets with Scapy (which has functions to manually craft abnormal packets with the desired field values), and use *iptables*, in multiple Oracle VirtualBox virtual machines running Ubuntu

10.04 Server. Two 'attacker' VMs send packets to a 'target server' VM. In a real-life scenario, attackers target a server on ports that are in the *LISTEN* state, to bring down the service.

Now, let's look at this in detail.

The attackers' configurations

My three Ubuntu Server VMs are connected through the VirtualBox 'Hostonly' network adapter. The target server is 192.168.56.102; .101 and .103 are the attackers. I am using Scapy 2.2.0. Going forward, extract the Scapy source, and as the *root*, run *python setup.py install*. Run *Scapy* with the command *scapy*.

To attack the target server (.102), insert the following *iptables* rules in the respective attacker VMs:

```
root@ubiserv:~# iptables -A OUTPUT -p tcp -s 192.168.56.101
--tcp-flags RST RST -j DROP
root@ubiserv:~# iptables -A OUTPUT -p tcp -s 192.168.56.103
--tcp-flags RST RST -j DROP
```



Note: This rule will DROP packets from the OUTPUT chain that have the RST flag set. The *iptables* rules will only apply to the kernel stack layer, not the application layer—so it will not apply to packets generated by Scapy, which creates the entire packet in its space. However, the malformed/manipulated packets crafted by Scapy will be seen by the kernel, which will send RST responses (resets) to the target, since it (the attacker's kernel) didn't initiate this TCP communication. To prevent this, we should use the above *iptables* rules, so that the kernel's RSTs will not get to the target—otherwise, the target's SYN buffer will not get full, and the DDOS attack will fail.